STT + API Gateway Deployment via Rubix

1. Scope and deployment model

The gateway runs in Rubix/Kubernetes. The STT API, the STT worker and the vLLM Whisper server run as Docker containers on the GPU host. Docker is the runtime for STT, not just an image builder; Rubix manages only the gateway.

Workload	Name	Runtime
API Gateway	`hope-api-gateway`	Rubix / Kubernetes
STT API	`hope-stt`	Docker container on the GPU host
STT worker	`hope-stt-worker`	Docker container on the GPU host
vLLM Whisper	`vllm-whisper`	Docker container on the GPU host

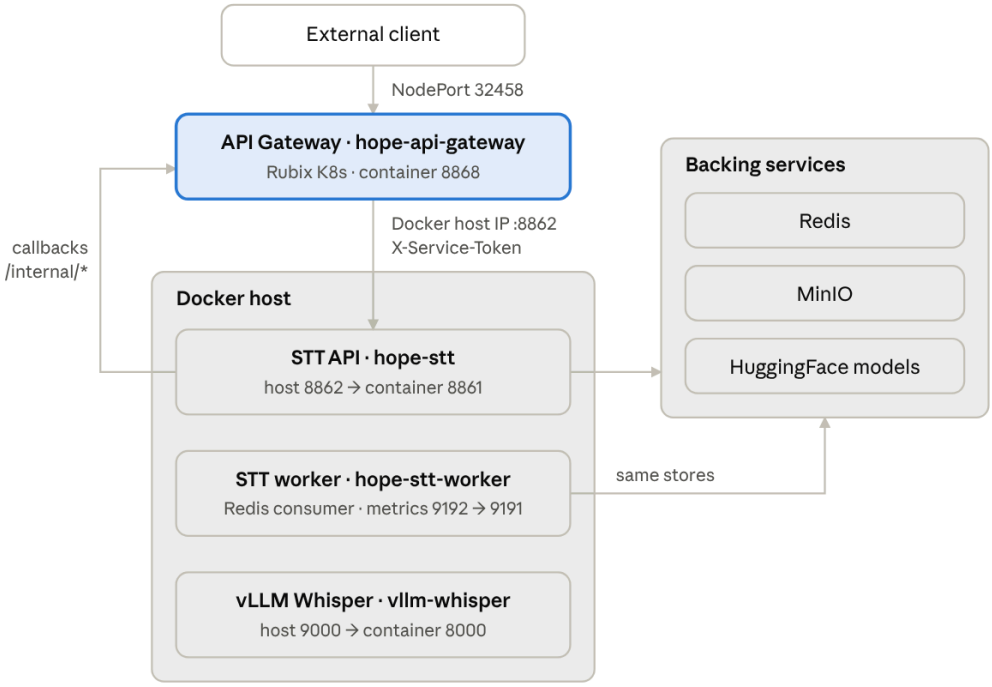
Deployment model:

- Gateway: the image is built with Docker and pushed to the registry configured in Rubix. Rubix creates/updates the workload, env vars, service, ports, probes and resources.
- STT API and worker: run directly on the Docker host, with env, port mappings, healthchecks and GPU access set in Docker env/compose.
- Gateway ↔ STT calls use host IP addresses and ports (STT host port 8862, gateway NodePort 32458), not Kubernetes service DNS.
- External access to the gateway uses NodePort.
- `kubectl` is not part of this workflow. Use Rubix logs/events/terminal for the gateway and `docker logs` for STT. CLI commands are optional diagnostics only.

Note: The checked-in `arca-deployment-template` manifests are a reference, not the deployment source. They set `replicas: 0`, use `envFrom` with Secret/ConfigMap names, and include ingress/Argo assumptions that may not exist in Rubix. Their STT parts only matter if STT later moves to Rubix. Do not paste them blindly.

2. Topology and ports

Only the gateway is exposed to clients; STT, the worker and vLLM run in Docker



Deployment topology · **3 workloads, 3 backing services**

External clients reach only the gateway's NodePort, `http://192.168.112.6:32458`. The gateway calls STT on the Docker host with `X-Service-Token`. STT and the worker call back to the gateway's `/internal/*` routes with `X-Internal-Service-Key`. The worker is a Redis consumer with no HTTP server.

Workload	Runtime	Container port	Reachable address
API Gateway	Rubix K8s	8868	`hope-api-gateway-svc:8866`, NodePort 32458
STT API	Docker	8861	`<DOCKER_HOST_IP>:8862`
STT worker	Docker	9191	`<DOCKER_HOST_IP>:9192` (metrics)
vLLM Whisper	Docker	8000	`<DOCKER_HOST_IP>:9000`

Cross-runtime calls use these addresses:

- Gateway → STT: `http://<DOCKER_HOST_IP>:8862`
- STT → Gateway: `http://192.168.112.6:32458/api/v1`

`STT_V2_URL` must not have `/api/v1`. `API_GATEWAY_URL` must have `/api/v1`.

STT runs outside the cluster, so it reaches the gateway through the NodePort address, not the service DNS name. Do not use `localhost` or pod IPs for these calls.

3. Build and push images

Build from the repository root. The gateway image is pulled by Rubix; the STT images run on the Docker host.

3.1 API Gateway (Rubix)

```
docker build \
  -f apps/api/Dockerfile \
  -t 192.168.112.13:5000/hope-v2/api:<tag> .

docker push 192.168.112.13:5000/hope-v2/api:<tag>
```

Rubix pulls the image from `192.168.112.13:5000`. The current gateway image is `192.168.112.13:5000/hope-v2/api:v1`. The image runs as uid 1001 and **exposes 8868**. Its entrypoint/CMD is already correct; do not override it in Rubix.

3.2 STT API (Docker host)

The STT API is a GPU workload, so build the `ml-runtime` target:

```
docker build \
  -f apps/stt/docker/Dockerfile \
  --target ml-runtime \
  -t hope-v2/stt-ml-runtime:<tag> .
```

The current image is `hope-v2/stt-ml-runtime:sha-dacdc311e`. It runs from the local image on the Docker host: build it there, or build elsewhere and bring it over with `docker pull` or an image export/import. The image command includes a CUDA preflight and **uvicorn on 8861**.

Do not use the `runtime` target for the GPU API: it is CPU-only and dev-oriented. `hope-python-base` is only needed when building the CPU `runtime` target, not `ml-runtime` or `worker`.

3.3 STT worker (Docker host)

```
docker build \
  -f apps/stt/docker/Dockerfile \
  --target worker \
  -t hope-v2/stt-worker:<tag> .
```

The current image is `hope-v2/stt-worker:latest`. The worker command runs the CUDA preflight and Dramatiq. It has no HTTP server.

4. Rubix gateway configuration, plus Docker STT configuration

The gateway is configured in Rubix. STT and the worker are configured on the Docker host; they have no Rubix workload, service, probe or GPU request.

4.0 Gateway (Rubix)

Field	Value
Deployment	`hope-api-gateway`
Image	`192.168.112.13:5000/hope-v2/api:v1`
Replicas	3
Container port	8868
Service	`hope-api-gateway-svc`, NodePort
Service port	8866 → 8868
NodePort	32458
External URL	`http://192.168.112.6:32458`
GPU	None
CPU request / limit	2000m / 2000m
Memory request / limit	4Gi / 4Gi
Persistent cache	No
Probes	HTTP

4.1 STT API and worker (Docker)

Field	STT API	STT worker
Container	`hope-stt`	`hope-stt-worker`
Image	`hope-v2/stt-ml-runtime:sha-dacdc311e`	`hope-v2/stt-worker:latest`
Port mapping	8862:8861	9192:9191 (metrics)
GPU	Docker GPU flags	Docker GPU flags
Persistent cache	None currently	None currently
Health	Docker HEALTHCHECK	Docker HEALTHCHECK

- No Rubix workload, service, probe or GPU request for STT or the worker.
- Both Docker healthchecks currently report healthy.

- No persistent cache volume is configured (see section 6).
- Gateway: NodePort 32458, service port 8866, target port 8868. Do not use 30088.

4.2 Probes

Gateway probes stay in Rubix:

Probe	Path
Startup	`/api/v1/health/startup`
Readiness	`/api/v1/health/ready`
Liveness	`/api/v1/health/live`

STT API: Docker HEALTHCHECK; no Rubix probe. The health paths are `/api/v1/health` and `/api/v1/health/live`. STT startup can take up to 15 minutes on a cold model cache, so keep any startup timeout generous.

4.3 vLLM Whisper (Docker)

vLLM serves `openai/whisper-large-v3-turbo` on the same Docker host and GPU as STT. It is not managed by Rubix.

Field	Value
Container	`vllm-whisper`
Image	`vllm/vllm-openai:latest`
Port mapping	9000:8000
GPU	`--runtime nvidia --gpus all`, `--ipc=host`
Cache	host `/data/cache_store/hf_global_cache` → `/root/.cache/huggingface`
Model	local snapshot of `openai/whisper-large-v3-turbo` (`41f01f3f`…), served as `openai/whisper-large-v3-turbo`
GPU memory	`--gpu-memory-utilization 0.1`
Limits	`--max-num-seqs 512`, `--max-model-len 448`, 1 audio per prompt

Run command:

```
docker run -d \
  --name vllm-whisper \
  --runtime nvidia \
  --gpus all \
  --ipc=host \
  -p 9000:8000 \
  -v /data/cache_store/hf_global_cache:/root/.cache/huggingface \
  --entrypoint bash \
```

```
  vllm/vllm-openai:latest \
  -c "pip install --no-cache-dir 'vllm[audio]' && exec vllm serve
/root/.cache/huggingface/hub/models--openai--whisper-large-v3-turbo/snapshots/41f01f3
fe87f28c78e2fbf8b568835947dd65ed9 \
  --served-model-name openai/whisper-large-v3-turbo \
  --gpu-memory-utilization 0.1 \
  --max-num-seqs 512 \
  --max-model-len 448 \
  --limit-mm-per-prompt '{\"audio\": 1}'"
```

Notes:

- The container runs `pip install` of `vllm[audio]` on every start. It needs outbound package-index access and adds startup time.
- The image tag is `latest`, so a recreated container can pull a different vLLM version. Pin a version or digest.
- `vllm serve` points at a local snapshot path, so the model must already be in the mounted cache.
- Port 9000 binds to 0.0.0.0; restrict it.
- No gateway or STT env var for vLLM is defined in this runbook. Add one when that integration is specified.

5. Environment variables

Gateway env lives in Rubix: use secret env for credentials and plain env for non-sensitive configuration, and do not put tokens in a plain ConfigMap. STT and worker env is passed through Docker env/compose; keep tokens out of version control.

5.1 API Gateway (Rubix)

Required:

```
NODE_ENV=production
PORT=8868
SECRETS_PROVIDER=vault
VAULT_ADDR=http://vault-store-svc.project-hope-vox-infra.svc.cluster.local:8200
VAULT_ROLE_ID=...
VAULT_SECRET_ID=...
DATABASE_URL=...
DB_CONNECTION_STRING=...
REDIS_URL=redis://:PASSWORD@REDIS_HOST:6379/0
STT_V2_URL=http://<DOCKER_HOST_IP>:8862
URL=http://192.168.112.6:32458
ADMIN_CONSOLE_URL=http://192.168.112.6:32540
```

Notes:

- `STT_V2_URL` must not end with `/api/v1`. It is missing from the current gateway env; add it.
- `DB_CONNECTION_STRING` should match `DATABASE_URL` if the image still reads the legacy name.
- With `SECRETS_PROVIDER=vault`, the gateway reads platform secrets such as `INTERNAL_ACCESS_TOKEN`, `JWT_SECRET_KEY` and MinIO credentials from Vault.
- If Rubix cannot provide AppRole access, use `SECRETS_PROVIDER=env` and supply every required secret as env. Do not set `vault` with incomplete AppRole values.
- The gateway does not need `API_GATEWAY_KEY` to call STT. That key is for STT → gateway. Do not use it as the gateway → STT token.
- If Rubix injects Vault values as env, make sure it injects resolved values, not paths like `secret/data/...`.

5.2 STT API (Docker)

Required:

```
NODE_ENV=production
DEPLOYMENT_ENVIRONMENT=production
HOST=0.0.0.0
PORT=8861
INTERNAL_ACCESS_TOKEN=...
API_GATEWAY_URL=http://192.168.112.6:32458/api/v1
API_GATEWAY_KEY=...
REDIS_URL=redis://:PASSWORD@REDIS_HOST:6379/0
MINIO_ENDPOINT=minio-host:9000
MINIO_SECURE=false
MINIO_ACCESS_KEY=...
MINIO_SECRET_KEY=...
MINIO_AUDIO_BUCKET=hope-audio
MINIO_CHUNK_BUCKET=hope-audio-chunks
STT_PROVIDER=whisper
HF_HOME=/home/hope/.cache/huggingface
HUGGINGFACE_CACHE_DIR=/home/hope/.cache/huggingface/hub
HF_XET_CACHE=/tmp/huggingface/xet
```

Optional:

```
LOG_LEVEL=INFO
DEBUG=false
CORS_ORIGINS=[]
MODEL_CACHE_MAX_MODELS=2
MODEL_CACHE_TTL_SECONDS=3600
TRANSCRIPTION_CHUNK_LENGTH_S=30
TRANSCRIPTION_TIMEOUT_SECONDS=600
API_GATEWAY_TIMEOUT=30
SARVAM_API_KEY=...
HUGGINGFACE_TOKEN=...
OMP_NUM_THREADS=8
MKL_NUM_THREADS=8
TORCH_NUM_THREADS=8
OTEL_SERVICE_NAME=hope-stt-v2
```

Critical:

- `REDIS_URL` and `MINIO_ENDPOINT` must be reachable from the Docker host. Do not copy Kubernetes service DNS unless the Docker host can resolve it.
- `INTERNAL_ACCESS_TOKEN` must reach STT as an env var. STT does not read Vault directly.
- `API_GATEWAY_KEY` must reach STT as an env var. It must be the raw value of an active `SERVICE_ACCOUNT` API key in the database, with scope `['*']` or at least `internal:stt:worker`.
- `NODE_ENV=production` is mandatory. If it is missing, STT treats the process as local and, with an empty token, bypasses inbound auth.
- `API_GATEWAY_URL` includes `/api/v1`.
- `MINIO_ENDPOINT` must not include `http://` or `https://`.
- `MINIO_SECURE` must match the endpoint scheme (`false` here).
- STT does not use `DATABASE_URL`; do not copy the manifest's `STT_V2_DATABASE_URL`.
- Do not set `HF_HUB_OFFLINE=1` unless the cache is fully pre-seeded.

5.3 STT worker (Docker)

Same core env as the STT API, except:

- `HOST`/`PORT` are not used for an HTTP server.
- `INTERNAL_ACCESS_TOKEN` is optional: the worker has no HTTP API, so it is only needed if you share one env set.
- `API_GATEWAY_KEY` is required for worker → gateway callbacks.

Worker-specific:

```
NODE_ENV=production
DEPLOYMENT_ENVIRONMENT=production
API_GATEWAY_URL=http://192.168.112.6:32458/api/v1
API_GATEWAY_KEY=...
REDIS_URL=redis://:PASSWORD@REDIS_HOST:6379/0
MINIO_ENDPOINT=minio-host:9000
MINIO_SECURE=false
MINIO_ACCESS_KEY=...
MINIO_SECRET_KEY=...
MINIO_AUDIO_BUCKET=hope-audio
MINIO_CHUNK_BUCKET=hope-audio-chunks
HF_HOME=/home/hope/.cache/huggingface
HUGGINGFACE_CACHE_DIR=/home/hope/.cache/huggingface/hub
HF_XET_CACHE=/tmp/huggingface/xet
WORKER_CONCURRENCY=4
WORKER_MAX_RETRIES=3
OTEL_SERVICE_NAME=hope-stt-v2-worker
```

Metrics, if enabled:

```
dramatiq_prom_host=0.0.0.0
dramatiq_prom_port=9191
```

`dramatiq_prom_port=9191` is the port inside the container; the host metrics port is 9192.

5.4 Vault vs Rubix/Docker env

Recommended split:

Value	Gateway (Rubix)	STT API (Docker)
`INTERNAL_ACCESS_TOKEN`	Vault or secret env	Docker env
`API_GATEWAY_KEY`	Not required	Docker env
`DATABASE_URL`	Secret env / Vault	Not used
`REDIS_URL`	Secret env	Docker env
MinIO credentials	Vault/env if used	Docker env
URLs, bucket names, HF paths	Plain env	Docker env
`NODE_ENV`	Plain env	Docker env

`INTERNAL_ACCESS_TOKEN` and `API_GATEWAY_KEY` are different credentials:

- `INTERNAL_ACCESS_TOKEN` → `X-Service-Token` on gateway → STT.
- `API_GATEWAY_KEY` → `X-Internal-Service-Key` on STT → gateway.

Both must match the same values on the receiving side. Rotate them together and restart the gateway in Rubix and the STT/worker containers in Docker.

6. Model cache and GPU

6.1 Model cache

The current Docker deployment has no persistent HF cache. The cache at `/home/hope/.cache/huggingface/hub` is inside the container and is lost when the container is recreated. This can cause long cold starts and repeated model downloads.

Recommended: mount the same host directory or named volume at that path in both `hope-stt` and `hope-stt-worker`, and set:

`vllm-whisper` already mounts a host cache at `/data/cache_store/hf_global_cache`. That directory is a candidate for the shared STT cache, provided uid 1001 can write to it.

```
HF_HOME=/home/hope/.cache/huggingface
HUGGINGFACE_CACHE_DIR=/home/hope/.cache/huggingface/hub
HF_XET_CACHE=/tmp/huggingface/xet
```

Requirements:

- The directory must be writable by uid 1001.
- The API and worker should share the cache.

Future Kubernetes migration only:

- With `emptyDir`, models redownload on every restart; cold starts can exceed the startup probe.
- With `hostPath`, pin the workload to the node that owns the cache.

6.2 GPU

The STT API and worker use Docker GPU access. Verify with `nvidia-smi` and `docker inspect`, and confirm the GPU allocation for `hope-stt`, `hope-stt-worker`, `vllm-whisper` and `tritonserver`.

The Kubernetes GPU request below is only relevant if STT moves to Rubix later:

```
requests:
  nvidia.com/gpu: "1"
limits:
  nvidia.com/gpu: "1"
```

The gateway has no GPU requirement.

7. Deployment order

Deploy the gateway before STT: STT can boot without the gateway, but gateway warmup and control-plane settings should be ready first.

1. **Redis**: reachable and password known.
2. **MinIO**: reachable; create `hope-audio`, `hope-audio-chunks` and `hope-models` if needed.
3. **Vault**: unsealed; KV, AppRole and policy ready for the gateway.
4. **Database schema**: run the database migration job if the gateway's schema is not current. The gateway image does not run migrations automatically.
5. **Env and secrets**: gateway in Rubix; STT and worker in Docker env/compose.
6. **API Gateway**: deploy in Rubix; verify Vault boot and health.
7. **STT API**: start the Docker container `hope-stt`; verify MinIO/Redis startup.
8. **STT worker**: start the Docker container `hope-stt-worker`; verify Dramatiq and metrics.
9. **End-to-end test** through the gateway.

8. Verification

Endpoints to check:

Component	Address
Gateway	`http://192.168.112.6:32458`
STT API	`http://<DOCKER_HOST_IP>:8862`
Worker metrics	`http://<DOCKER_HOST_IP>:9192/metrics`
vLLM Whisper	`http://<DOCKER_HOST_IP>:9000`

8.1 Gateway

Use Rubix logs and events. Check:

- Vault login succeeds.
- No warmup misses for `INTERNAL_ACCESS_TOKEN`, `JWT_SECRET_KEY`, etc.
- All 3 pods are ready.
- NodePort 32458 is reachable from outside.

```
curl -sS http://192.168.112.6:32458/api/v1/health/live
curl -sS http://192.168.112.6:32458/api/v1/health/ready
```

8.2 STT API

From `docker logs hope-stt`, confirm:

- Initializing the MinIO client succeeds.
- Redis connects.
- No "INTERNAL_ACCESS_TOKEN is unset" message in a deployed environment.
- No repeated model download errors.
- The Docker healthcheck reports healthy.

From a Rubix pod/terminal:

```
curl -sS http://<DOCKER_HOST_IP>:8862/api/v1/health
```

From the Docker host:

```
curl -sS http://127.0.0.1:8862/api/v1/health
curl -sS http://127.0.0.1:8862/api/v1/health/live
```

Health routes are exempt from auth. A protected route without `X-Service-Token` must return 401; if it returns 200, auth is bypassed or the environment is wrong.

8.3 STT worker

Check `docker logs hope-stt-worker` for Dramatiq startup and heartbeat, and confirm the Docker healthcheck is healthy. Scrape metrics from the Docker host:

```
curl -sS http://127.0.0.1:9192/metrics
```

The worker has no HTTP health endpoint; do not configure an HTTP probe on 8861.

8.4 End-to-end

1. Send a short transcription through the gateway's public NodePort, `http://192.168.112.6:32458`.
2. Confirm gateway → STT (`<DOCKER_HOST_IP>:8862`) returns a response.
3. Confirm STT can call back to gateway `/internal/*` at `192.168.112.6:32458` using `API_GATEWAY_KEY`.
4. Confirm the worker consumes a queued batch job.
5. Confirm audio/chunk objects appear in MinIO.
6. Confirm the model cache populates.

8.5 vLLM Whisper

Check `docker logs vllm-whisper` for a completed model load, then from the Docker host:

```
curl -sS http://127.0.0.1:9000/v1/models
```

The response should list `openai/whisper-large-v3-turbo`.

Note: If a bad token caused 401s, fix the token pair rather than disabling auth:

- `INTERNAL_ACCESS_TOKEN` must match gateway Vault/env and STT env.
- `API_GATEWAY_KEY` must match the active database API key used by STT.

12. Quick reference

Item	Value
Gateway deployment	`hope-api-gateway`
Gateway service	`hope-api-gateway-svc`
Gateway image	`192.168.112.13:5000/hope-v2/api:v1`
Gateway container port	8868
Gateway service port	8866
Gateway NodePort	32458
Gateway URL	`http://192.168.112.6:32458`
STT API	`hope-stt` (Docker)
STT image	`hope-v2/stt-ml-runtime:sha-dacdc311e`
STT host port	8862 → 8861
STT health	`http://<DOCKER_HOST_IP>:8862/api/v1/health`
STT worker	`hope-stt-worker` (Docker)
Worker image	`hope-v2/stt-worker:latest`
Worker metrics	9192 → 9191
vLLM Whisper	`vllm-whisper` (Docker)
vLLM image	`vllm/vllm-openai:latest`
vLLM host port	9000 → 8000
vLLM model	`openai/whisper-large-v3-turbo`
vLLM cache	`/data/cache_store/hf_global_cache`
Gateway → STT	`http://<DOCKER_HOST_IP>:8862`
STT → Gateway	`http://192.168.112.6:32458/api/v1`
Gateway → STT header	`X-Service-Token: INTERNAL_ACCESS_TOKEN`
STT → Gateway header	`X-Internal-Service-Key: API_GATEWAY_KEY`

Item	Value
Audio bucket	`hope-audio`
Chunk bucket	`hope-audio-chunks`
Cache mount	None (current, STT)
STT database	None